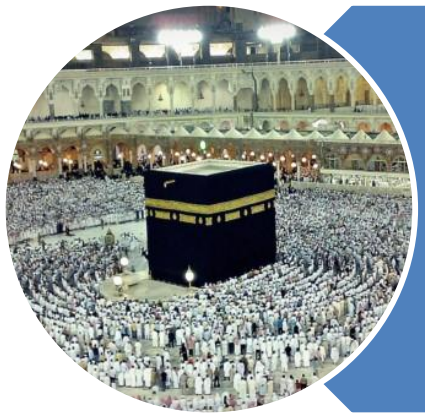
Data Structures - I

CPCS 204

رَبِّ اشْرَحْ لِي صَدْرِي وَيَسِّرْ لِي أَمْرِي وَاحْلُلْ عُقْدَةً مِنْ لِسَانِي يَفْقَهُوا قَوْلِي



O my Lord! Expand for me my chest [with assurance] and ease for me my task and untie the knot from my tongue that they may understand my speech. **(Quran 20 : 25-28)**

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

CPCS 204

Sorting Techniques

Implementation

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

Introduction

- Techniques – Quadratic - **$O(n^2)$**
  - Selection Sort
  - Insertion Sort
  - Bubble Sort
- Techniques – Logarithmic - **O(n log n)**
  - Merge Sort
  - Quick Sort

CPCS 204

Quadratic Sorting

Selection Sort

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

Selection Sort - **Algorithm**

1) Find the minimum/maximum value in the list of n elements
   - Search from index 0 to index n-1
2) Swap that minimum value with the value in the first position
   - At index 0
3) Repeat steps 1 and 2 for the remainder of the list

Selection Sort - Implementation

```
public void SelectionSort(int arr[]) {
    int n = arr.length;
    for (int i = 0; i < n-1; i++){
      int min_idx = i;
      for (int j = i+1; j < n; j++)
          if (arr[j] < arr[min_idx])
                min_idx = j;
      int temp = arr[min_idx];
      arr[min_idx] = arr[i];
      arr[i] = temp;
    }
}
```

CPCS 204

Quadratic Sorting

Insertion Sort

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

Insertion Sort - **Algorithm**

- Descending Order
  - Starting with the 2nd element,
  - continually SWAP it with the previous element
  - Until
    - it found a bigger value **OR**
    - It reaches to the first index i.e. "0"

Insertion Sort - Implementation

```
public void InsertSort(int arr[]) {
    int n = arr.length;
    for (int i = 1; i < n; ++i) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] < key) {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = key;
    }
}
```

CPCS 204

Quadratic Sorting

Bubble Sort

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

Bubble Sort - **Algorithm**

- You always compare consecutive elements
  - Going left to right
- Whenever two elements are out of place,
  - SWAP them
- At the end of a single iteration,
  - the maximum element will be in the last spot
- Now you simply repeat this n times
  - where n is the number of elements being sorted

Bubble Sort - Implementation

```
public void BubbleSort(int arr[]) {
      int n = arr.length;
      for (int i = 0; i < n-1; i++)
          for (int j = 0; j < n-i-1; j++)
              if (arr[j] > arr[j+1]){
                  int temp = arr[j];
                  arr[j] = arr[j+1];
                  arr[j+1] = temp;
              }
}
```

CPCS 204

Logarithmic Sorting

Merge Sort

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

CPCS 204

Merge Sort - **Algorithm**

```
method MergeSort(array) {
    1. MergeSort(left half of array)
    2. MergeSort(right half of Array)
    3. Merge
}
```

Merge Sort - Implementation

```
public void MergeSort(int values[], int start, int end) {
    int mid;
    if (start < end) {

        mid = (start+end)/2;

        MergeSort(values, start, mid);

        MergeSort(values, mid+1, end);

        Merge(values, start, mid+1, end);
    }
}
```


Dr. Jonathan (Yahya) Cazalas

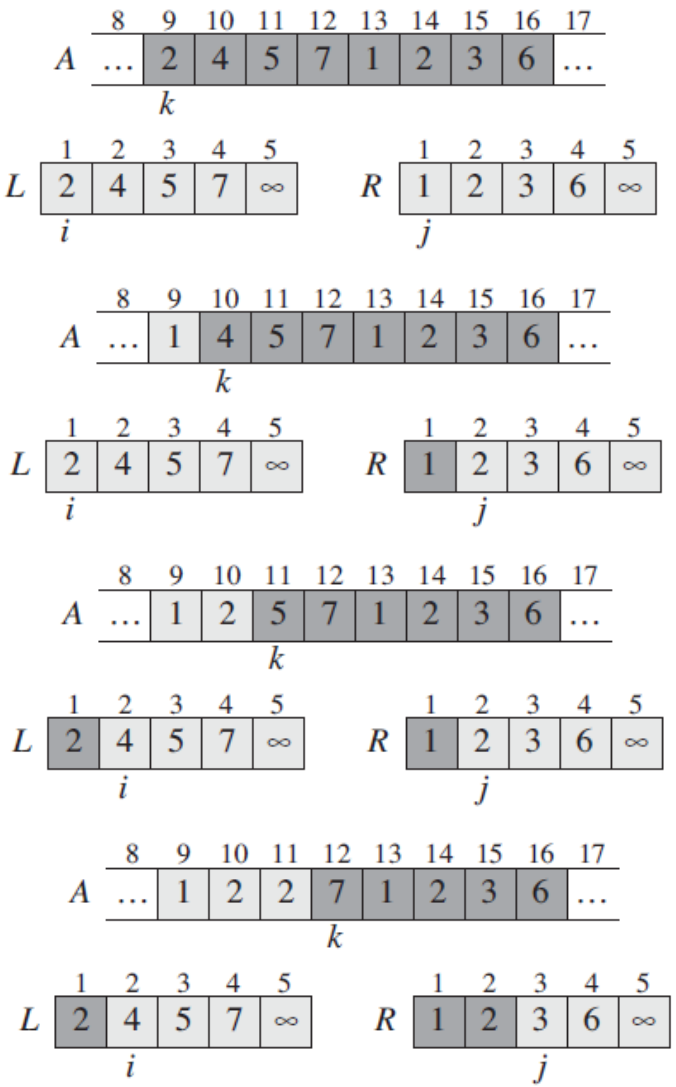
Dr. Muhammad Umair Ramzan

CPCS 204

Merge Sort - Merge

MERGE(A, p, q, r)

```
1  n1 = q − p + 1
2  n2 = r − q
3  let L[1..n1 + 1] and R[1..n2 + 1] be new arrays
4  for i = 1 to n1
5      L[i] = A[p + i − 1]
6  for j = 1 to n2
7      R[j] = A[q + j]
8  L[n1 + 1] = ∞
9  R[n2 + 1] = ∞
10 i = 1
11 j = 1
12 for k = p to r
13     if L[i] ≤ R[j]
14         A[k] = L[i]
15         i = i + 1
16     else A[k] = R[j]
17         j = j + 1
```



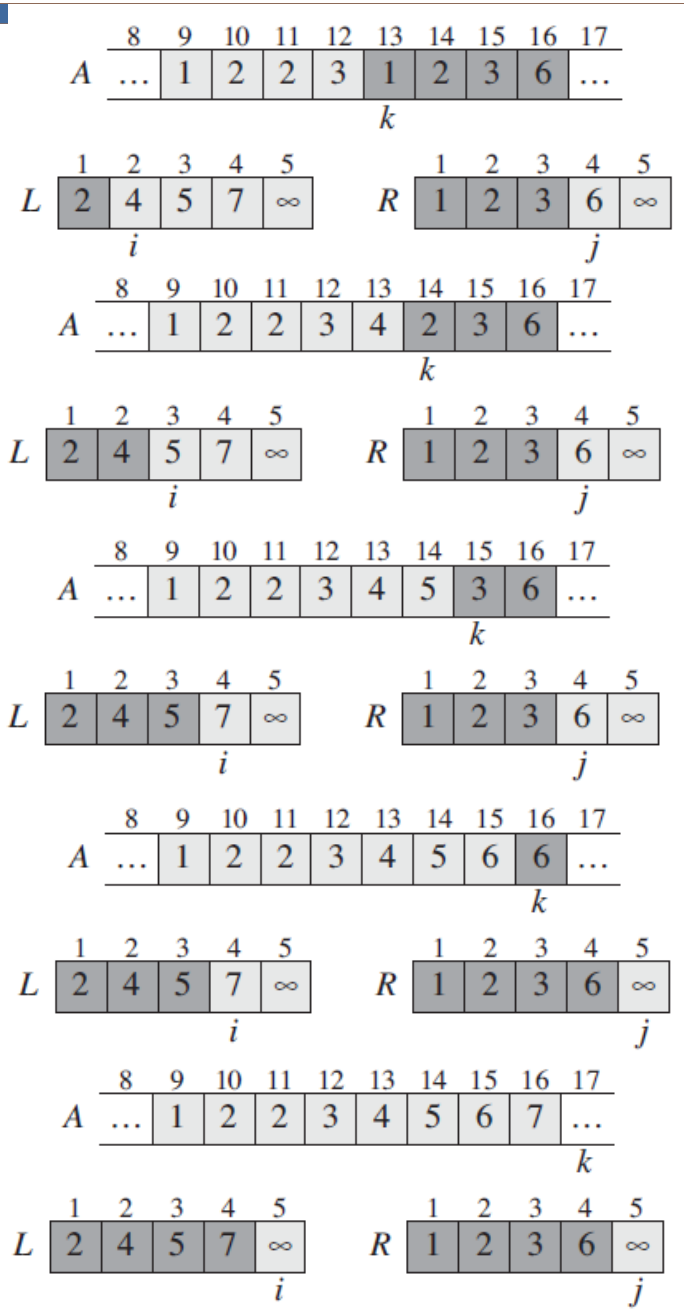
Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

Merge Sort - Merge

MERGE(A, p, q, r)

```
1  n1 = q − p + 1
2  n2 = r − q
3  let L[1..n1 + 1] and R[1..n2 + 1] be new arrays
4  for i = 1 to n1
5      L[i] = A[p + i − 1]
6  for j = 1 to n2
7      R[j] = A[q + j]
8  L[n1 + 1] = ∞
9  R[n2 + 1] = ∞
10 i = 1
11 j = 1
12 for k = p to r
13     if L[i] ≤ R[j]
14         A[k] = L[i]
15         i = i + 1
16     else A[k] = R[j]
17         j = j + 1
```



Dr. Jonathan (Yahya) Cazalas

CPCS 204

Logarithmic Sorting

Quick Sort

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

Quick Sort - **Algorithm**

```
method QuickSort(array) {
    1. partition the arrays
    2. QuickSort(smaller elements partition)
    3. QuickSort(larger elements partition)
}
```

Quick Sort - Implementation

```
public void quickSort(int numbers[], int low, int high) {

    if (low < high) {

        int pivotIdx = partition(numbers, low, high);

        quickSort(numbers, low, pivotIdx-1);

        quickSort(numbers, pivotIdx+1, high);
    }
}
```

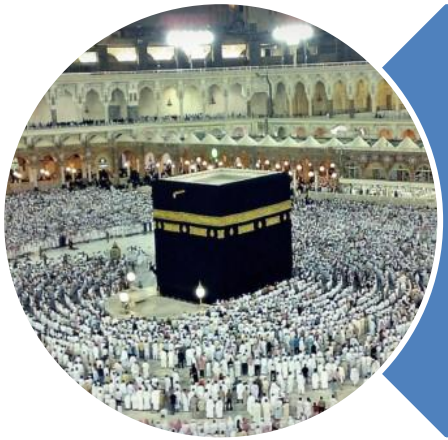
Quick Sort - Partition

```
public int partition(int vals[], int low, int high) {
    int temp, randomIndex;
    int pivotIdx;
    if (low == high) return low;

    randomIndex = low + (int)(Math.random()*(high-low+1));
    temp = vals[randomIndex];
    vals[randomIndex] = vals[low];
    vals[low] = temp;

    pivotIdx = low;
    low++;
    while (low <= high) {
       while (low <= high && vals[low] <= vals[pivot]) low++;
       while (high >= low && vals[high] > vals[pivot]) high--;
       if (low < high)
          swap(vals, low, high);
    }
    swap(vals, pivotIdx, high);
    return high;
}
```

Data Structures - I

"No Bonus" marks for this course anymore

CPCS 204



Allah (Alone) is Sufficient for us, and He is the Best Disposer of affairs (for us).(Quran 3 : 173)

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan